Universidade do Minho

Ano Letivo: 2025/26

Mineração de Dados

Text Mining & NLP II

Miguel Rocha | mrocha@di.uminho.pt
Diana Ferreira | diana@di.uminho.pt

Contents

1 Syntactic Analysis: Parsing

2 Similarity Measures

3 WordNet

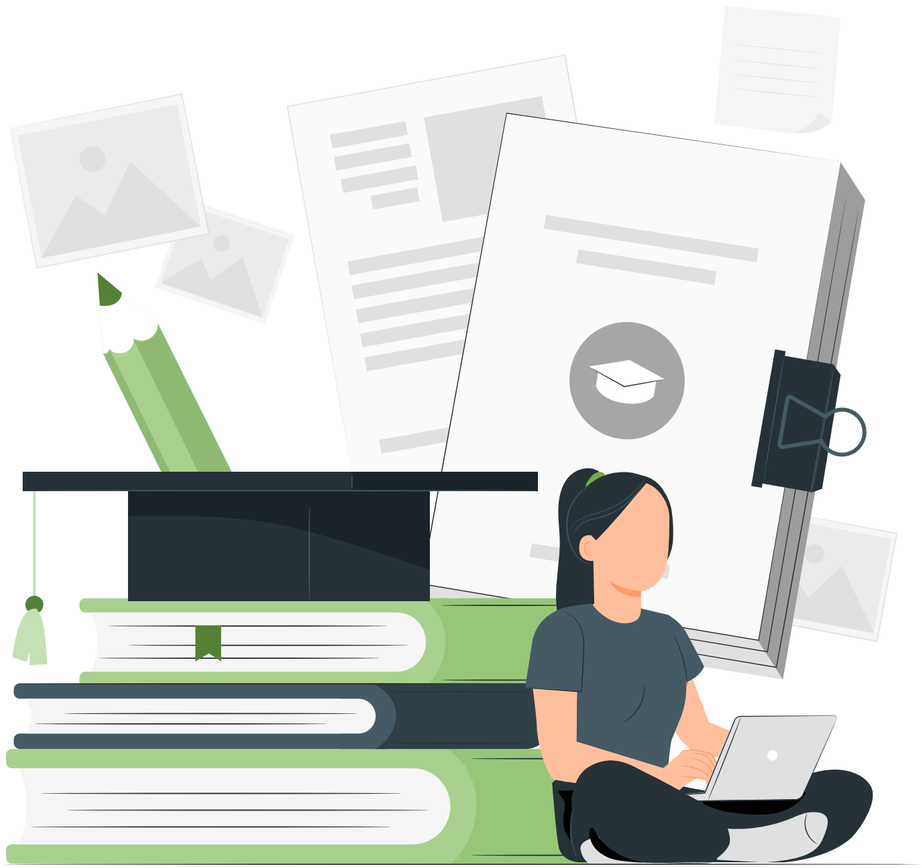
4 Word Embeddings

Learning Objectives

By the end of this lecture, students should be able to:

- Implement a simple search system using Vector Space Model (VSM) and cosine similarity
- Explore semantic relationships using WordNet
- Extract grammatical relationships using Dependency Parsing
- Manipulate word embeddings and explore semantic properties

Bibliography



Books:

1. Vajjala, S., Majumder, B., Gupta, A., & Surana, H. (2020). Practical Natural Language Processing. O'Reilly
2. Geetha, T.V. (2024). Understanding Natural Language Processing – Machine Learning and Deep Learning Perspectives. Pearson.

Official Documentation:

1. NLTK Documentation: https://www.nltk.org/
2. spaCy Documentation: https://spacy.io/
3. Gensim Documentation: https://radimrehurek.com/gensim/
4. WordNet: https://wordnet.princeton.edu/
5. GloVe Embeddings: https://nlp.stanford.edu/projects/glove/

Overview of last week concepts

➡ Preprocessing Techniques

Technique	Description	Example	Applications
Tokenization	Split text into tokens	"I love NLP" → ["I", 'love', "NLP"]	Preprocessing for all NLP tasks
Stemming	Reduce words to their root form (may not be a real word)	"liked," "liked," "will like" → 'like'	Search, recommendation systems
Lemmatization	Reduce words to the base form in the dictionary	"better" → "bett"	Semantic analysis, chatbots
Stop Words	Remove common words with no meaning	"the," "a," "of," "to," "with" (removed)	Classification, clustering
PoS Tagging	Tagging part of speech	"cat" → NOUN, "runs" → VERB	Syntactic understanding
NER	Identifying named entities	"John went to Lisbon" → [John:PERSON, Lisbon:LOC]	Information extraction

Stemming vs. Lemmatization

Stemming → fast, crude, maximizes recall
Lemmatization → slower, precise, maximizes linguistic correctness

When Stemming is better than Lemmatization?

1. When speed is critical

This matters when processing:

- millions of documents
- real-time queries
- large search indexes

2. When approximate matching is acceptable (or desirable)

In search engines, it is often better to return too many results than miss relevant ones.

Advantage of stemming: All documents with any form of the word are found! Lemmatization also finds them (depending on implementation), but it is slower.

3. When processing large datasets

Example domains:

- search engines
- document retrieval
- log analysis
- web indexing

Overview of last week concepts

➡ Feature Extraction Techniques

Technique	Description	Example	Applications
Bag of Words	Word count vector	[cat:1, mouse:1, dog:0, ...]	Simple classification
TF-IDF	Weight terms by overall importance	Rare words have greater weight	Search, summarization
N-grams	Capture word sequences	“I don't like” (bigram) maintains negation	Sentiment analysis

Limitations of Classical Representations:

- ✕ Sparse vectors with very high dimensionality.
- ✕ They do not capture semantics or relationships between words.
- ✕ No sense of context: ‘right’ (correct) = ‘right’ (direction)
- ✕ No treatment of synonyms or polysemy

NLP Pipeline

The **NLP Pipeline** is a sequence of steps used to transform raw text into structured information that machines can analyze.

Raw Text
↓
Text Cleaning
↓
Tokenization
↓
NLP Tasks (PoS Tagging, NER, etc.)
↓
Stopword Removal
↓
Stemming / Lemmatization
↓
Feature Extraction
↓
Machine Learning Model

Remove irrelevant characters, HTML, punctuation
Example: remove tags from scraped web pages

Split text into smaller units (tokens)

Extract useful information:
- Part of Speech (PoS) Tagging
- Named Entity Recognition (NER)
- Dependency Parsing

Remove common words without significant meaning, such as: the, is, in, at, of

Reduce words to their base form
Example:
running → run; better → good

Convert text into vector representation
Example:
BoW
TF-IDF
N-grams
Embeddings

Key Principle: Garbage in, garbage out - preprocessing is critical!

Kahoot

Goal

- To **reward and compensate students** who actively attend and participate in classes
- To reinforce key concepts in an engaging and competitive way

Rules

- Only students **physically present in class** may participate
- No external help (notes, phones, AI tools, or collaboration)
- Any form of cheating results in **disqualification**

Scoring & Rewards

🥇 **1st place**: +0.5 bonus points

🥈 **2nd place**: +0.3 bonus points

🥉 **3rd place**: +0.2 bonus points

(Bonus points are added to the continuous assessment component.)

⚠️ Bonus points are **non-transferable** and **cannot exceed course limits**

The Problem of Order and Relationships

Parsing

"The cat ate the mouse."

If I know that 'Cat' is a noun and 'Ate' is a verb, do I know who ate whom?

→ POS Tagging identifies the **category**, but **not** the **function**.

How can we capture the structure of who acts on whom?

To understand the meaning, we need to map the **dependency relationships** between words.

CONSTITUENCY PARSING

Represents the syntactic structure of a sentence by grouping words into hierarchical **phrases (constituents)**.

DEPENDENCY PARSING

Represents grammatical structure as **binary relations between words** (head → dependent).

The Problem of Order and Relationships

Parsing

❌ ***"The cat ate the mouse."***

💡 To understand the meaning, we need to map the **dependency relationships** between words.

CONSTITUENCY PARSING

```
[S
   [NP
      [Det The]
      [N   cat]
   ]
   [VP
      [V  ate]
      [NP
         [Det the]
         [N   mouse]
      ]
   ]
]
```

S → Sentence
NP → Noun Phrase (The cat)
VP → Verb Phrase (ate the mouse)
The second **NP** is the object of the verb

- **Key idea:** A sentence is built from nested phrases.
- Tree represents phrase hierarchy

DEPENDENCY PARSING

```
     ate
    /   \
  cat    mouse
  |        |
 The      the
```

ate is the ROOT (main verb of the sentence)
cat is the nominal subject → *nsubj*
mouse is the direct object → *obj*
The and **the** are determiners → *det*

- The structure is built over **words**, not phrases.
- The main verb is usually the structural center of the sentence.
- Each word (except ROOT) has exactly **one head**.

⚠️ Dependency parsing is more computationally efficient and better at handling free-order languages (such as Portuguese).

The Problem of Order and Relationships

➡ What is Dependency Parsing?

Dependency Parsing examines the **grammatical structure** of a sentence to **identify the relationships**, or **dependencies**, between **words** in a sentence.

- Unlike POS Tagging, which classifies words individually, Dependency Parsing reveals how words are connected, building direct links between individual words. Each word depends on another word that acts as its head.

- Dependendy Parsing represents these connections through a dependency tree or graph, which makes it easier for machines to understand meaning and extract who is doing what to whom.

The Problem of Order and Relationships

➡ Dependency Parsing Key Concepts

1) **The Root:** Usually the **main verb** of the sentence. It is the point from which everything emanates.

2) **Head and Dependent:** The relationship is asymmetrical. The Head is the "master" word, the Dependent is the one that adds information.

3) **Arcs:** The arrows that connect the words, always labeled with the grammatical function.

Universal Dependencies – most common tags:

Tag	Meaning	Example	Dependency
root	Main predicate of sentence	John wrote the document.	wrote
nsubj	Nominal subject	The cat ate the mouse.	ate → cat
obj	Direct object	She read the book.	read → book
iobj	Indirect object	She gave him a gift.	gave → him
amod	Adjectival modifier	He has a red car.	car → red
advmod	Adverbial modifier	The mouse ran quickly.	ran → quickly
nmod	Nominal modifier	The roof of the house.	roof → house
det	Determiner	The kid eats carrots.	kid → the

https://universaldependencies.org/

The Problem of Order and Relationships

➡ Dependency Parsing with spaCy

STEP 1: Parse a Sentence

```python
import spacy
from spacy import displacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Example sentence
doc = nlp("The cat quickly ate the mouse.")

# Print dependency relations
for token in doc:
    print(token.text, token.dep_, token.head.text)
```

OUTPUT

```
The det cat
cat nsubj ate
quickly advmod ate
ate ROOT ate
the det mouse
mouse dobj ate
. punct ate
```

The Problem of Order and Relationships

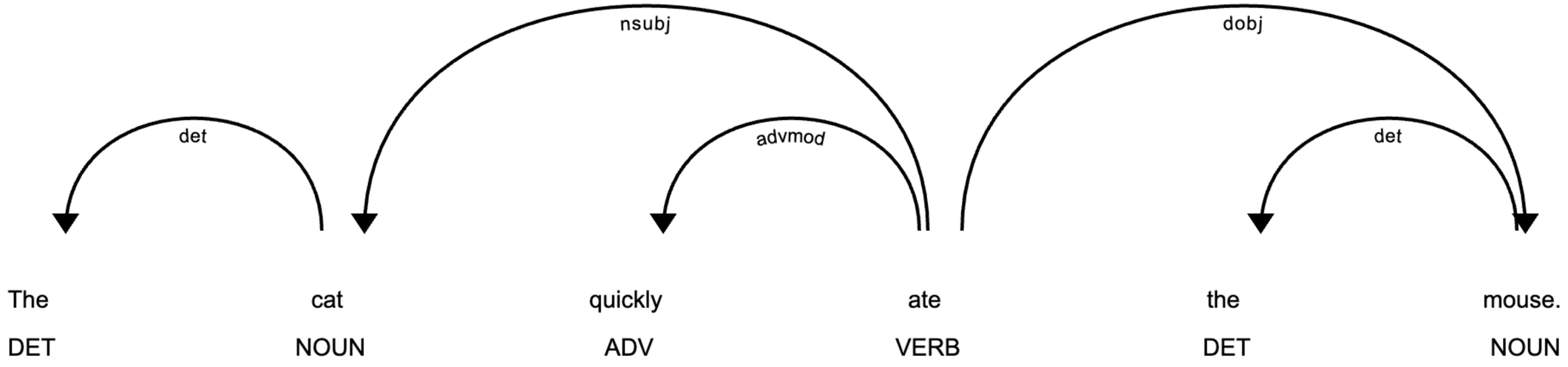
➡ Dependency Parsing with spaCy

STEP 2: Generate a Dependency Tree

```
displacy.render(doc, style="dep", jupyter=True)

# Or save as HTML
html = displacy.render(doc, style="dep")
with open("dependency_tree.html", "w", encoding="utf-8") as f:
    f.write(html)
```

OUTPUT



The Problem of Order and Relationships

➡ Dependency Parsing Applications

Relationship Extraction

Problem: Extract structured knowledge from unstructured texts

Example: "João bought an Apple iPhone in 2023"

Parsing identifies:

Subject: João (who)

Verb: bought (action)

Object: iPhone (what)

Modifier: Apple (origin/manufacturer)

Modifier: 2023 (when)

⚠ **Usefulness:** Construction of knowledge graphs, semantic search

The Problem of Order and Relationships

➡ Dependency Parsing Applications

Aspect-Based Sentiment Analysis

Problem: Overall sentiment is not enough; we want sentiment by aspect.

Example: "Review: "The cell phone battery is great, but the screen is terrible."

Parsing identifies:

"great" (positive sentiment) → linked to "battery"

"terrible" (negative sentiment) → linked to "screen"

Result:

Cell phone (overall): neutral/mixed

Battery: positive

Screen: negative

⚠ **Usefulness:** Analysis of product reviews, detailed feedback

The Problem of Order and Relationships

➡ Dependency Parsing Applications

Question and Answer (QA) Systems

Problem: Without parsing: Bag-of-words model would not know who performed certain action

Example: Text: “Mary killed John with a knife.” Question: “Who killed John?”

Parsing identifies:

Verb: killed

Subject: Maria

Object: João

Match with question:

Question searches for “who” = subject of the action “kill”

Answer: “Mary”

The Problem of Order and Relationships

➡ Dependency Parsing: The Limits of Syntax

CHALLENGES:

- ❌ **Languages with free word order:** Portuguese allows for confusing variations
- ❌ **Structural ambiguity:** "I saw the man with a telescope" (who has the telescope?)
- ❌ **Very long sentences:** Errors accumulate
- ❌ **Training dependency:** Models need annotated corpora
- ❌ **Specific domains:** Performance drops in technical/legal/medical texts

The Problem of Order and Relationships

➡ Dependency Parsing: The Limits of Syntax

❌ **Problem:** Syntax resolves structure, but not always **meaning**.

Example: "He touched her palm."

palm can mean:

- The inside of the hand
- A palm tree

To solve the problem of **meaning (semantics),** we need something beyond trees and rules:

we need **Mathematics and Vectors**!

Analyzing similarities

Problem: How to compare documents? How to capture relationships between words?

In BoW, the words “Car” and “Vehicle” are orthogonal (perpendicular) vectors.

Doc 1: “The car is fast.”
Doc 2: “The driver crashed the vehicle.”
Doc 3: “The road was full of curves.”

Car = [1, 0, 0]

Vehicle = [0, 1, 0]

For the computer, they are as different as “Car” and “Dog.” <u>There is no notion of synonymy.</u>

Analyzing similarities

➡ Vector Space Model (VSM)

- Document A: "The cat ate the mouse."
- Document B: "The mouse was eaten by the cat."

❌ **Problem: How can we mathematically measure whether A and B are "similar"?**

↓

The Vector Space Model (VSM) is an algebraic model for representing text documents as **vectors**.

- Each document or word is a point in an *N*-dimensional space, where N is the size of the vocabulary.
- Each word in the vocabulary is a dimension.
- The value in that dimension is how many times the word appears (or TF-IDF).

Analyzing similarities

➡ Vector Space Model (VSM)

EXAMPLE:
- Doc1 → “hello”
- Doc2 → “world world”
- Doc3 → “hello world”

STEP 1: Define vocabulary
Vocabulary: [hello, world]

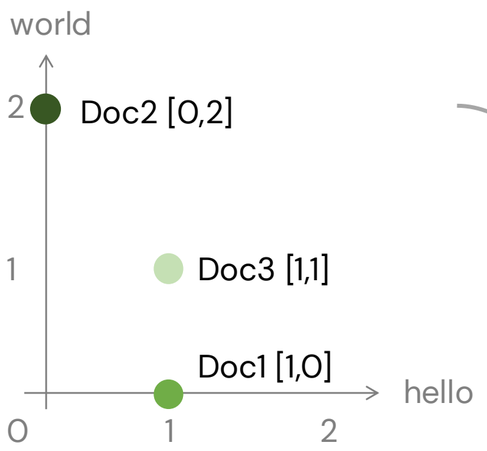
STEP 2: Count words to create vectors

Document	hello	world
Doc1:	1	0
Doc2:	0	2
Doc3:	1	1

Now, the documents have become vectors:

- Doc1 → [1, 0]
- Doc2 → [0, 2]
- Doc3 → [1, 1]

STEP 3: Plot in 2-dimensional space
These are our axes:
Y-axis → world
X-axis → hello



- Each document is a point in the 2Dspace.
- **Doc1 and Doc2** are far apart → no shared words

- Words → axes
- Documents → points
- Similar documents → closer in space

Analyzing similarities

➡ Vector Space Model (VSM)

- Document A: "The cat ate the mouse."
- Document B: "The mouse was eaten by the cat."

STEP 1: Vocabulary: [the, cat, ate, mouse, was, eaten, by] → So we have a 7-dimensional space.

STEP 2:
Doc A → [2, 1, 1, 1, 0, 0, 0]
Doc B → [2, 1, 0, 1, 1, 1, 1]

Both sentences mean **almost the same thing,** but their vectors are different.

STEP 3:
In 2D or 3D, axis order is easy to visualize.

But in reality:
- We do not have 2 dimensions.
- We may have 10,000+ dimensions (one per word)

At that point:
- The axis labels do not matter visually → Only the math matters!

Analyzing similarities

➡ Measuring Similarity in Vector Space

Once text is represented as vectors, we need a way to measure **how similar two documents are**.

How to Measure the Distance Between Vectors?

Option 1: **Euclidean Distance**
Measures the "straight-line" distance between two points.

Formula: $\sqrt{[(x_1 - x_2)^2 + (y_1 - y_2)^2 + \ldots]}$

Small distance → documents are similar
Large distance → documents are different

Example:
A = [1, 1]
B = [10, 10]

$$d = \sqrt{(1-10)^2 + (1-10)^2}$$
$$= \sqrt{(-9)^2 + (-9)^2}$$
$$= \sqrt{81 + 81}$$
$$= \sqrt{162}$$
$$\approx 12.73$$

❌ **Problem:** Sensitive to magnitude (document size)

- Long documents are very "far" from short documents, even if they deal with the same subject.

💡 They point in the same direction BUT distance is large. Euclidean distance penalizes document length.

Analyzing similarities

➡ Measuring Similarity in Vector Space

Once text is represented as vectors, we need a way to measure **how similar two documents are**.

How to Measure the Distance Between Vectors?

Option 2: **Cosine Similarity**
Measures the cosine of the angle between vectors, not the distance.

Formula: $$\text{Cosine Similarity} = \cos(\theta) = \frac{A \cdot B}{||A||\,||B||}$$

Where:
$A \cdot B$= dot product
$||A||$= vector magnitude

- Focuses on direction, not size
- Ranges from -1 to 1 (or 0 to 1 for text):
  - θ = 0° → same direction → similarity = 1 (identical)
  - θ = 90° → orthogonal → similarity = 0 (completely different)

Example:
A = [1, 1]
B = [10, 10]

Dot Product *A . B = 1x10 + 1x10 = 20*

Magnitude $$||A|| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

$$||B|| = \sqrt{10^2 + 10^2} = \sqrt{200}$$

$$\sqrt{200} = 10\sqrt{2}$$

Final Calculation $$\cos(\theta) = \frac{20}{\sqrt{2} \times 10\sqrt{2}} = \frac{20}{20} = 1$$

💡 Cosine similarity = 1 → Because they point in the same direction.

Analyzing similarities

➡ Measuring Similarity in Vector Space

Aspect	Euclidean Distance	Cosine Similarity
What it measures	Straight-line distance between vectors	Angle between vectors
Focus	Magnitude + direction	Direction only
Sensitive to document length?	Yes	No
Range of values	$0 \rightarrow \infty$	$-1 \rightarrow 1$
Identical vectors	Distance = 0	Similarity = 1
Orthogonal vectors	Large distance	Similarity = 0
Works well for text?	Usually not ideal	Yes (most common in NLP)

Euclidean → measures distance

"How far apart are they?"

Cosine → measures orientation

"Are they talking about the same thing?"

In text:
- Documents vary in length
- The focus is on shared terms, not size

Analyzing similarities

VSM and Cosine Similarity

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = ["cat sat on mat", "dog sat on log"]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

# Cosine similarity between all document pairs
cos_sim = cosine_similarity(X)
print(cos_sim)
```

Advantages

- Simple and intuitive
- Works well for document ranking
- Scalable

Limitations

- Data sparsity
- Ignores word order
- Vocabulary dependent
- Ignores semantics

```
OUTPUT:
[[1. 0.33609693]
[0.33609693 1. ]]
```

Diagonal (1.0) → each document is 100% similar to itself.
0.34 → similarity between Doc1 and Doc2.
They share some words (sat, on), but also have different words (cat vs dog, mat vs log).

Analyzing similarities

➡ Cosine Similarity Applications

Plagiarism Detection

Suspicious document → vector A

Source document → vector B

Strategies:

Overall similarity > 0.8 → possible plagiarism

Similarity by paragraphs → localized plagiarism

Demonstration:

Original: “Artificial intelligence transforms the world”

Plagiarism: “The world is transformed by artificial intelligence”

Cosine with BoW = 1.0 (same words!)

Analyzing similarities

➡ Cosine Similarity Applications

Search Engine / Information Retrieval
User query → vector Q
Each document in database → vector D

Strategy:
- Compute cosine similarity between Q and all documents
- Rank documents by highest similarity
- Return Top-K results

Demonstration:
Query: "machine learning applications"
Document A: "applications of machine learning in healthcare"
Document B: "cooking recipes for beginners"

Cosine(Q, A) → high
Cosine(Q, B) → near 0

✔ Document A appears first in search results.

Analyzing similarities

➡ The Limits of Cosine Similarity

CHALLENGES:

❌ **Ignores Word Meaning (Semantics):** Cosine with BoW only compares surface words.

❌ **Cannot Handle Synonyms:**

Doc A: "The car is fast."

Doc B: "The automobile is quick."

Cosine similarity with BoW → low

- "car" ≠ "automobile"
- "fast" ≠ "quick"

Even though semantically they are very similar.

❌ **Sensitive to Vocabulary Mismatch:**

If two documents talk about the same topic (AI vs. Artificial Intelligence) but use different terminology → similarity drops.

❌ **Ignores Word Order**

Example:

- "Dog bites man."
- "Man bites dog."

Cosine with BoW = **1.0**

But meanings are completely different.

The problem of semantics

➡ WordNet – A Network of Words and Meanings

What is it? It is a **large lexical knowledge base** of English developed at Princeton University.

- Created by George Miller (1985) – cognitive psychologist
- Organized as a semantic network – Instead of organizing words alphabetically, it organizes them by **meaning**.
- Different from a dictionary: Not only **defines**, but also **relates** words
- Size ~155 k words organized into ~117 k synsets

Core Idea: Words are grouped into sets of synonyms called **Synsets** (synonym sets). Each synset represents **one concept**.

All words in the synset are interchangeable in a given context.

EXAMPLE – Synset for *car*:

- car
- automobile
- auto
- motorcar

The problem of semantics

➡ WordNet – A Network of Words and Meanings

How is it structured?

WordNet connects **synsets** through **semantic relations.**

- **Synonymy:** Words with similar meaning (car – automobile)
- **Antonymy:** Opposite meanings (hot – cold)
- **Hypernymy (is-a relationship):** A more general concept (cat → animal)
- **Hyponymy:** A more specific concept (animal → cat)

Example Hierarchy

→ animal
→ mammal
→ cat

→ This creates a **semantic network (graph structure)**.

The problem of semantics

➡ WordNet – A Network of Words and Meanings

```python
import nltk
nltk.download('wordnet’) # Download only the first time
from nltk.corpus import wordnet as wn

# Retornar synsets de ‘car'
for synset in wn.synsets(‘car'):
    print(synset.name() , "→", synset.definition())

# Retornar sinónimos de ‘car'
car = wn.synset('car.n.01')
print([lemma.name() for lemma in car.lemmas()])

# Similaridade entre 'car' e 'automobile'
auto = wn.synset('automobile.n.01')
print(f"Similaridade: {car.path_similarity(auto)}")
```

Advantages

- Organizes words by meaning
- Separates word senses
- Encodes semantic relations
- Allows similarity via hierarchy

Limitations

- Manually built
- Static
- Limited vocabular
- Similarity based only on tree distance

```
OUTPUT:
car.n.01 → a motor vehicle with four wheels; usually propelled by an internal combustion engine
car.n.02 → a wheeled vehicle adapted to the rails of railroad
car.n.03 → the compartment that is suspended from an airship and that carries personnel and the cargo and the power plant
car.n.04 → where passengers ride up and down
cable_car.n.01 → a conveyance for passengers or freight on a cable railway
['car', 'auto', 'automobile', 'machine', 'motorcar']
Similaridade: 1.0
```

The problem of semantics

➡ WordNet Applications

Word Sense Disambiguation

Problem: Determining which sense of a word is being used.

Critical for machine translation, semantic search, etc.

Example:

Sentence 1: “I went to the bank to deposit money.”
Sentence 2: “I sat on the park bench.”

Meaning bank1: {bank, financial institution}
Meaning bank2: {bank, seat, chair}

How WordNet helps: Context + hierarchical relationships

The problem of semantics

➡ WordNet Applications

Measure similarity between concepts using WordNet

Minimum Path Metric (Path Similarity):

The shorter the path between synsets in the hierarchy, the more similar they are.

Sim (car, automobile) = 1.0 (same synset)

Sim (car, motorcycle) = 0.5 (common parent: vehicle)

Sim (car, idea) = 0.1 (long path)

The problem of semantics

➡ WordNet Limitations

CHALLENGES:

- ❌ **Limited coverage:** Neologisms, technical terms, slang
- ❌ **Manual maintenance:** Does not scale with language evolution
- ❌ **Does not capture context dynamically:** Context does not always resolve ambiguity
- ❌ **English-centricity:** WordNet for Portuguese exists (WordNet.PT), but is smaller
- ❌ **Lack of nuance:** Binary relationships, without weights or probabilities.

Summary - What We Have Learned So Far

Technique	What it captures	Limitation
BoW/TF-IDF	Topics, frequency	Ignores synonyms and order
Dependency Parsing	Syntactic relationships, who-does-what	Ignores word meaning
WordNet	Semantic relationships, synonyms	Manual, non-contextual

Problem: How to automatically combine the best of all worlds?

Word Embeddings -> next week!

Kahoot

Goal

- To **reward and compensate students** who actively attend and participate in classes
- To reinforce key concepts in an engaging and competitive way

Rules

- Only students **physically present in class** may participate
- No external help (notes, phones, AI tools, or collaboration)
- Any form of cheating results in **disqualification**

Scoring & Rewards

🥇 **1st place**: +0.5 bonus points

🥈 **2nd place**: +0.3 bonus points

🥉 **3rd place**: +0.2 bonus points

(Bonus points are added to the continuous assessment component.)

⚠️ Bonus points are **non-transferable** and **cannot exceed course limits**

Practical Application

➡ **Worksheet PL5**

Blackboard